

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS2DA: Data Structures & Algorithms

Divide & Conquer

Recap: Algorithms, Hash Tables and Quicksort

An algorithm is a well-defined sequence of steps for solving a problem.

Hash tables use arrays and linked lists to give a map with **fast** operations.

Quicksort uses **divide & conquer** to sort an array **quickly**.

Today

What exactly does **fast** or **quickly** mean? How can we tell without a computer?

Two most important strategies in algorithm design:

- **divide & conquer** — today
- **dynamic programming** — second half of module

Recap: Divide & Conquer

We **divide** the problem of **sorting one big array** into smaller subproblems that we can **conquer**.

We do this **recursively** until we get a tiny **base case** problem that we can solve immediately.

Then we **combine** the results of the subproblems to get the solution to the whole problem.

Minimum/Maximum

Minimum/Maximum — Obvious Algorithm

```
Algorithm: minimum(A)
Input: Array of values A[0..n-1]
Output: Smallest value in A

m = A[0]
for k from 1 to n-1
    if A[k] < m
        m = A[k]
return m
```

Uses $n - 1$ comparisons.

To find maximum, `maximum(A)`, change `A[k] < m` to `A[k] > m`.

To find **both** minimum and maximum, use both algorithms: $2n - 2$ comparisons.

Can we do better?

Minimum/Maximum — D&C

Algorithm: minmax(A)

Input: Array of values A[0..n-1]

Output: Pair: minimum and maximum value in A[0..n-1]

```
if size(A) == 1
    return (A[0], A[0])
if size(A) == 2                -- Trick 1: Only 1 compare for max AND min
    if A[0] < A[1]              of 2 items
        return (A[0], A[1])
    else
        return (A[1], A[0])
(L, R) = split(A)
(minl, maxl) = minmax(L)
(minr, maxr) = minmax(R)
return (min(minl, minr), max(maxl, maxr)) -- Trick 2: Only 2 compares
                                         to "merge" max AND min

where min(x,y) = if x < y then x else y
      max(x,y) = if x > y then x else y
```

Try it.

Analysis — How Many Comparisons?

Define a **recurrence relation** $T(n)$ — a function that calculates the "cost" of the algorithm on input of size n , in terms of the cost on smaller inputs:

$$T(1) = 0$$

$$T(2) = 1$$

$$T(n) = 2T\left(\frac{n}{2}\right) + 2$$

The recurrence relation solves to:

$$T(n) = \frac{3}{2}n - 2$$

This is better than $2n - 2$.

Do we care?

Big-O Notation

Comparing Algorithms

Ultimately, for input of size n , we want to be able to predict:

- How much **time** (in **seconds**) the algorithm takes.
- How much **space** (in **bytes**) the algorithm takes.

So we can make an informed choice about which algorithm to use **for our application**.

We can certainly measure the time taken by a particular algorithm on a particular input with a particular computer, but:

- Measurements can be noisy — how many experiments do we need?
- Is it fair to compare quicksort on a 1980s PC vs slowsort on a 2020 PC?
- Performance varies with n .
 - What if slowsort wins on $n=5$, but quicksort wins on $n = 5,000,000$?
 - I want a nice summary of performance without looking at a graph/table.

Big-O — Intuition

To compare absolute performance of algorithms A and B:

- Draw graphs of time/space vs input size on same axes.
- If A's line on the graph is always below B's, A is clearly better.

But we want a way of comparing algorithms that ignores:

- CPU speed/memory size of the computer
- small inputs

So, to compare algorithms A and B:

- Squash A's graph vertically (as much as you want).
- Cover up the left end of the graphs (as much as you want).
- If A's line on the graph is always below B's, A is at least as good as B.

Big-O — Notation

Suppose we have 2 functions $f(n)$ and $g(n)$, which tell us how much time/space algorithms A and B take on different inputs.

Suppose:

- Algorithm A: $f(n) = n$
- Algorithm B: $g(n) = n^2$

Clearly $f(n) \leq g(n)$, so A is better than B.

We write: $f(n) = \mathcal{O}(n^2)$

and say something like: "Algorithm A is (big) O n squared"

The "O" is short for "order". We could say: "A is order n squared."

Despite the notation, note that $=$ is not symmetric here!

We do **not** write: $\mathcal{O}(n^2) = f(n)$

Big-O — Formal Definition

If $f(n) = \mathcal{O}(g(n))$

then there exist positive constants c and n_0 such that:

for all $n \geq n_0$, $0 \leq f(n) \leq cg(n)$

n_0 is where we "cover up the left".

c is how much we squash $f(n)$, or stretch $g(n)$.

We also have:

- Ω : Lower bound: If $g(n) = \mathcal{O}(f(n))$ then $f(n) = \Omega(g(n))$
- Θ : Exact bound: If $f(n) = \mathcal{O}(g(n))$ and $g(n) = \mathcal{O}(f(n))$ then $f(n) = \Theta(g(n))$

Computational Model

We cannot easily estimate how long a program takes in seconds by looking at the source code.

So we approximate:

- All basic operations (arithmetic, primitive variable assignment, array lookup) take "constant" time: 1 or $\mathcal{O}(1)$.
- A loop with n iterations takes n times the length of the loop body.
- Function calls take as long as the function body.
- For recursive functions, we have to solve a **recurrence equation**.

Ignoring Lower Order Terms & Constant Factors

We can simplify by discarding lower order terms and constant factors:

$$\mathcal{O}(50n^3 + 100n^2 + 20n \log n + 1000000) = \mathcal{O}(n^3)$$

Hence:

Lester's Linear Slowdown Slogan

The algorithm that takes twice as long is just as fast.

Obvious minimum/maximum calculation: $2n - 2 = \mathcal{O}(n)$

Divide & conquer minimum/maximum calculation: $\frac{3}{2}n - 2 = \mathcal{O}(n)$

Do we care? No.

Big-O — Common Complexities

Name	Formula	Code Pattern	Example Algorithm
constant	1	simple statement	add two numbers
logarithmic	$\log n$	divide in half	binary search
linear	n	single loop	find the maximum
linearithmic	$n \log n$	divide and conquer	mergesort
quadratic	n^2	double loop	insertion sort
cubic	n^3	triple loop	matrix multiplication
exponential	2^n	exhaustive search	knapsack, TSP

Big-O — Common Complexities

When can you reasonably use algorithms of different complexities?

- $\mathcal{O}(n \log n)$ and below: Use as much as you like.
- $\mathcal{O}(n^2)$, $\mathcal{O}(n^3)$: OK to use non-interactively, or occasionally with small inputs.
- $\mathcal{O}(n^k)$ where k constant: Polynomial time — theoretically, this is the limit of "fast" or "efficient".
- $\mathcal{O}(2^n)$: Forget it unless n is tiny or you have very good heuristics for the average case.

You may have heard of the classes of problems **P** and **NP**:

- **P** is *polynomial* time.
- **NP** is *nondeterministic polynomial* time: Best algorithms known are worst-case exponential time.

Mergesort

Mergesort — Overview

Quicksort:

- Pick pivot p . Partition array into $<p, =p, >p$ — $\mathcal{O}(n)$.
- Recursively sort $<p, >p$.
- No need to combine: everything already in place.
- Total time: $\mathcal{O}(n \log n)$ with typical pivot choice; $\mathcal{O}(n^2)$ in worst case.

Top-down mergesort:

- Split array into equal-sized halves: left, right.
- Recursively sort left, right.
- Combine: **merge** left and right, maintaining sortedness — $\mathcal{O}(n)$.
- Total time $\mathcal{O}(n \log n)$ guaranteed as split always halves.

Mergesort — Merging

Input: Sorted arrays $L[0..m-1]$, $R[0..n-1]$

Output: Sorted merge array $M[0..m+n-1]$

```
l = 0, r = 0, i = 0      -- Iterate through arrays from left to right
while l < m or r < n    -- Keep going until both inputs are used up
    if r == n            -- Right is used up, so use left
        M[i] = L[l], increment i, l
    else if l == m        -- Left is used up, so use right
        M[i] = R[r], increment i, r
    else if L[l] <= R[r]  -- Left's next element is
        M[i] = L[l], increment i, l  -- smaller than right's, so use left
    else                  -- Otherwise, right's next element is
        M[i] = R[r], increment i, r  -- smaller than left's, so use right
return M
```

Try it.

Mergesort vs Quicksort

In practice, quicksort usually wins because it can be done **in-place** with minimal working space. **Array** operations are very fast. The "constant factors" are small.

We might sometimes prefer mergesort because:

- No random numbers needed to guarantee good performance.
- Mergesort may perform better on **linked lists**.
- Mergesort is **stable**: relative orderings of items with same key are preserved.
- Some aesthetic sense that mergesort is theoretically nicer/cleaner.

Order Statistics

Order Statistics: Selection

The k th order statistic of a list is the k th smallest element.

We have seen how to find maximum and minimum in $\mathcal{O}(n)$ time.

If we sort an array, the k th smallest element is in the k th position.

We can sort in $\mathcal{O}(n \log n)$ time and then find any order statistic in $\mathcal{O}(1)$ time.

$\mathcal{O}(n)$ is obviously optimal for maximum/minimum.

Can we do better than $\mathcal{O}(n \log n)$ in the middle?

Selection Sort

Input: Array $A[0..n-1]$

Output: Sorted array A

```
for i from 0 to n-2          -- Iterate over the whole array.
    min_val = A[i]
    min_idx = i
    for j from i+1 to n-1    -- Find the smallest unsorted value.
        if A[j] < min_val
            min_val = A[j]
            min_idx = j
    swap(A[i], A[min_idx])    -- Swap it into place as next sorted value.
return A
```

What is the time complexity?

Order Statistics with Selection Sort

Input: Array $A[0..n-1]$, k in range $[0..n-1]$

Output: k th smallest value in A

```
for i from 0 to k          -- Stop early.
    min_val = A[i]
    min_idx = i
    for j from i+1 to n-1
        if A[j] < min_val
            min_val = A[j]
            min_idx = j
    swap(A[i], A[min_idx])
return A[k]                -- Return the array element, not the array.
```

What is the time complexity?

We can even do this without modifying the array. (Take care over duplicate elements.)

Quickselect

To get the k th smallest element in the correct position, we only need to sort part of the array.

In selection sort, we put elements in the correct position from left to right. We can stop as soon as we have sorted up to the element we want.

In quicksort, the pivot ends up in the correct position.

If we know the element we want is (say) less than the pivot, we can just recurse on the left side.

The resulting modification of quicksort is **quickselect**.

Quicksort

Input: Array $A[0..n-1]$

Output: Sorted A

if size of $A \leq 1$ then return

pick a pivot p from A

partition A into 3 sections: " $< p$ ", " $= p$ ", " $> p$ "

sort section " $< p$ "

sort section " $> p$ "

Quickselect

Input: Array $A[0..n-1]$, k in $[0..n-1]$

Output: k th smallest element of A

if size of $A = 1$ then return $A[0]$

pick a pivot p from A

partition A into 3 sections: " $< p$ ", " $= p$ ", " $> p$ "

if $k < \text{size of } "< p"$ then return $\text{select}("< p", k)$

if $k < \text{size of } "< p" + \text{size of } "= p"$ then return p

return $\text{select}("> p", k - (\text{size of } "< p" + \text{size of } "= p"))$

Improving Quickselect — Random Pivot

We improved quicksort with random choice of pivot, to get $\mathcal{O}(n \log n)$ **expected** time instead of $\mathcal{O}(n^2)$ worst case.

We can improve quickselect with random choice of pivot, to get $\mathcal{O}(\log n)$ **expected** time — the same as a binary search — instead of $\mathcal{O}(n^2)$ worst case.

Try it.

Convex Hull

Convex Hull — Mergehull

Imagine hammering nails into a board, then putting an elastic band around the nails. The (filled) shape formed by the band is the convex hull.

Given a set of points P in 2D space, the points on the perimeter of the convex hull:

- are a **subset** of P
- define a hull of P — form corners of a **polygon** that contains all points in P
- are minimal — removing a point from the perimeter would mean some points lie outside the resulting polygon;
- are convex — every turn on the polygon is clockwise (or anticlockwise, if you go in the opposite direction)

How can we compute the convex hull efficiently?

Separate Upper and Lower Hulls

We know the left-most, right-most, upper-most and bottom-most points in P must be on the perimeter of the convex hull.

We can compute the lower part of the convex hull (beginning left-most and ending right-most) separately from the upper part (beginning right-most and ending left-most).

Input: Set of points P

Output: Ordered list of points on corners of perimeter of convex hull of P

```
lower = lowerhull(P)
```

```
upper = upperhull(P)
```

```
remove right-most point from lower
```

```
remove left-most point from upper
```

```
return concatenate(lower, upper)
```

Divide & Conquer Lower Convex Hull

We want to use divide & conquer to compute the lower convex hull.

Merging convex hulls is easier if they are disjoint.

So partition points around a pivot: split into points on left and points on right.

Input: Set of points P

Output: Ordered list of points on corners of lower convex hull of P

```
if size of  $P \leq 2$  return sort( $P$ )
```

```
pick a pivot  $p$ 
```

```
partition  $P$  into:
```

```
    left = points left of  $p$ , vertically below  $p$ ,  $p$  itself
```

```
    right = points vertically above  $p$ , right of  $p$ 
```

```
return merge_lower_hull(lowerhull(left), lowerhull(right))
```

Merging Lower Convex Hulls

We can merge left and right lower convex hulls by concatenation...

...but we may need to delete points in the middle to keep the result convex.

If 3 consecutive points form a clockwise turn, delete the middle point.

```
Input: ordered points on left lower convex hull L
       ordered points on right lower convex hull R
Output: ordered points of combined lower convex hull

repeat
    if last 2 points of L and first point of R form clockwise turn
        delete last point of L
    if last point of L and first 2 points of R form clockwise turn
        delete first point of R
until no more points deleted
return concatenate(L, R)
```

We can use vector cross product to calculate if a turn is clockwise.

Ensuring Good Pivot Choice

The algorithm so far looks somewhat like quicksort (partition around pivot) and mergesort (combine sorted subproblems with merge).

In quicksort, we needed good pivot choice for expected $\mathcal{O}(n \log n)$ complexity.

In practice, choosing randomly was often best. We can do the same here, but...

If we start the whole algorithm by sorting points from left to right (with mergesort), we can easily use the middle element as a perfect pivot.

Then the algorithm looks more like mergesort: we always split the list in half.

This guarantees $\mathcal{O}(n \log n)$ complexity.

"Faster" Integer Multiplication

Integer Multiplication: Russian Multiplication

Most processors multiply integers up to machine word size in hardware.

For larger binary integers, multiply using a long multiplication algorithm.

When multiplying large decimal numbers, we have to add large numbers and multiply by 0-9. For binary numbers, we only need to multiply by 0-1.

Input: Binary integers to multiply, x and y

Output: Product of x and y

```
z = 0
while x != 0
    if x & 1 = 1
        z = z + y
    shift x right 1 bit
    shift y left 1 bit
return z
```

-- Look at every bit of x in turn
-- Is Least Significant Bit of x equal to 1?
-- If so: $z = z + (y*1)$ -- $O(n)$ operation
-- Else LSB of x is 0: $z = z + (y*0)$
-- Examine the next bit of x
-- Arithmetic interpretation: $y = y * 2$

What is n?

How fast is this addition algorithm, assuming increment/decrement are $\mathcal{O}(1)$:

Input: Positive integers to add, x and y

Output: Sum of x and y

```
z = 0
while x != 0
    decrement x
    increment z
while y != 0
    decrement y
    increment z
```

Sometimes we need to be careful about exactly what "input of size n" means.

Can we do better than Russian multiplication? Yes, but to analyse the algorithm we will need the Master Theorem.

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

Next time: Master Theorem and Trees